

Aula 3: Padrões de Projeto e Exemplos Práticos

Prof. Lucas Ricardo Duarte Bruzzone

24 de Fevereiro de 2025

Agenda da Aula

- 1 Revisão: Padrões de Projeto
- 2 Padrões de Projeto por Categoria
 - Singleton (Criacional)
 - Decorator (Estrutural)
 - Strategy (Comportamental)
- 3 Próximos Passos
- 4 Exercícios Práticos

O que são Padrões de Projeto?

- São como "receitas" para resolver problemas comuns no desenvolvimento de software
- Ajudam a criar código que é:
 - Mais fácil de manter (quando precisa corrigir problemas)
 - Mais fácil de entender (outros desenvolvedores entendem melhor)
 - Mais flexível (quando precisa mudar ou adicionar funcionalidades)
- Classificados em 3 categorias:
 - **Criacionais:** Como criar objetos de forma organizada
 - **Estruturais:** Como organizar classes e objetos juntos
 - **Comportamentais:** Como os objetos se comunicam entre si

- **Criacionais:** Ajudam na criação de objetos
 - **Exemplo:** Fábrica de carros - diferentes maneiras de produzir carros
 - Singleton, Factory Method, Abstract Factory, Builder, Prototype
- **Estruturais:** Como juntar peças do sistema de forma eficiente
 - **Exemplo:** Adaptadores de tomada - conectar componentes diferentes
 - Adapter, Decorator, Facade, Composite, Proxy, Bridge, Flyweight
- **Comportamentais:** Como os objetos se comunicam e delegam responsabilidades
 - **Exemplo:** Regras de trânsito - protocolos de comunicação
 - Strategy, Command, Template Method, Iterator, State, Chain of Responsibility

- **Propósito:** Garantir que exista só uma instância de uma classe em toda a aplicação
- **Analogia do mundo real:**
 - Um país só pode ter um presidente por vez
- **Problema que resolve:**
 - Como garantir que todos os componentes do sistema acessem a mesma instância?
 - Como evitar criar múltiplos objetos que deveriam ser únicos?

Casos de Uso para Singleton

Exemplos práticos onde o Singleton é útil:

- **Conexão com banco de dados:** Evita abrir múltiplas conexões desnecessárias
- **Arquivo de configuração:** Todos acessam as mesmas configurações
- **Logger:** Centraliza todos os logs da aplicação em um único sistema
- **Impressora:** Controlar o acesso a um recurso físico compartilhado
- **Cache:** Todos os componentes compartilham o mesmo cache de dados

Implementação do Singleton - Explicada

```
1 class Singleton:
2     # Variável de classe para armazenar a única instância
3     __instance = None
4
5     @classmethod
6     def get_instance(cls):
7         # Se não existe uma instância, cria uma
8         if cls.__instance is None:
9             cls.__instance = Singleton()
10            print("Instância criada pela primeira vez")
11            # Retorna a instância (nova ou existente)
12            return cls.__instance
13
14    def __init__(self):
15        # Impede a criação direta de outras instâncias
16        if self.__instance is not None:
17            raise Exception("Erro! Use get_instance() para obter o Singleton")
18        self.__instance = self
19
20    def operation(self):
21        # Método de exemplo
22        print("Operação executada no Singleton")
```

Como Usar o Singleton - Passo a Passo

```
1 # FORMA CORRETA - Obter a inst ncia nica
2 print("Obtendo primeira refer ncia ao Singleton:")
3 singleton1 = Singleton.get_instance() # Cria e retorna a inst ncia
4
5 print("\nObtendo segunda refer ncia ao Singleton:")
6 singleton2 = Singleton.get_instance() # Retorna a mesma inst ncia
7
8 # Ambas as refer ncias apontam para o mesmo objeto
9 print("\nVerificando se s o a mesma inst ncia:")
10 print(f"singleton1 == singleton2? {singleton1 == singleton2}") # True
11
12 print("\nChamando m todo atrav s de diferentes refer ncias:")
13 singleton1.operation() # M todo chamado na primeira refer ncia
14 singleton2.operation() # M todo chamado na segunda refer ncia
15
16 # FORMA INCORRETA - Tentar criar uma inst ncia diretamente
17 # print("\nTentando criar inst ncia diretamente (vai gerar erro):")
18 # singleton3 = Singleton() # Erro: Use get_instance()
```


Vantagens

- Garante uma única instância
- Economiza recursos do sistema
- Ponto de acesso global e controlado
- Inicialização preguiçosa (só cria quando necessário)

Desvantagens

- Dificulta os testes unitários
- Cria dependências "escondidas" no código
- Pode causar problemas em sistemas com múltiplas threads
- Viola o princípio de responsabilidade única

- **Propósito:** Adicionar funcionalidades a objetos sem modificar seu código
- **Analogias do mundo real:**
 - Colocar coberturas em um sorvete (chocolate, granulado, calda)
 - Vestir camadas de roupa (camiseta + casaco + cachecol)
- **Problema que resolve:**
 - Como adicionar recursos a objetos específicos sem criar subclasses para cada combinação?
 - Como compor funcionalidades de forma flexível em tempo de execução?

Casos de Uso para Decorator

Exemplos práticos onde o Decorator é útil:

- **Formatação de texto:** adicionar negrito, itálico, sublinhado
- **Sistemas de streaming:** adicionar compressão, criptografia, bufferização
- **Interface gráfica:** adicionar bordas, rolagem, sombras a componentes
- **Notificações:** adicionar múltiplos canais (email, SMS, push)
- **Middleware em aplicações web:** autenticação, logging, compressão

Implementação do Decorator - Explicada

```
1 # Interface/Componente Base - Define opera es b sicas
2 class Texto:
3     def renderizar(self):
4         return "Texto simples" # Comportamento b sico
5
6 # Decorator Base - Mant m refer ncia ao componente envolvido
7 class TextoDecorator(Texto):
8     def __init__(self, texto):
9         self.texto_envolvido = texto # Guarda refer ncia ao componente
10
11     def renderizar(self):
12         # Por padr o , apenas repassa a chamada para o componente
13         return self.texto_envolvido.renderizar()
14
15 # Decorators Concretos - Cada um adiciona sua funcionalidade
16 class NegritoDecorator(TextoDecorator):
17     def renderizar(self):
18         # Adiciona tags de negrito ao resultado do componente
19         return "<b>" + self.texto_envolvido.renderizar() + "</b>"
20
21 class ItalicoDecorator(TextoDecorator):
22     def renderizar(self):
23         # Adiciona tags de it lico ao resultado do componente
24         return "<i>" + self.texto_envolvido.renderizar() + "</i>"
```

Como Usar o Decorator - Passo a Passo

```
1 # Criando o componente base
2 texto_simples = Texto()
3 print("Texto original:")
4 print(texto_simples.renderizar())
5 # Sa da: "Texto simples"
6
7 # Adicionando um decorator (negrito)
8 print("\nTexto com negrito:")
9 texto_negrito = NegritoDecorator(texto_simples)
10 print(texto_negrito.renderizar())
11 # Sa da: "<b>Texto simples</b>"
12
13 # Adicionando mais um decorator (it lico sobre o negrito)
14 print("\nTexto com negrito E it lico:")
15 texto_negrito_italico = ItalicoDecorator(texto_negrito)
16 print(texto_negrito_italico.renderizar())
17 # Sa da: "<i><b>Texto simples</b></i>"
18
19 # Criando outra combina o (apenas it lico)
20 print("\nTexto apenas com it lico:")
21 texto_italico = ItalicoDecorator(Texto())
22 print(texto_italico.renderizar())
23 # Sa da: "<i>Texto simples</i>"
```

Vantagens

- Adiciona funcionalidades sem modificar código existente
- Permite combinar comportamentos de forma flexível
- Evita hierarquias de herança extensas
- Permite adicionar ou remover responsabilidades em tempo de execução

Desvantagens

- Pode criar muitos objetos pequenos
- Código pode ficar mais difícil de entender
- Remover um decorator específico pode ser complicado
- A ordem de aplicação dos decorators pode importar

Strategy: Explicação Simples

- **Propósito:** Definir uma família de algoritmos intercambiáveis
- **Analogias do mundo real:**
 - Escolher o meio de transporte (carro, ônibus, bicicleta)
 - Método de pagamento em uma loja (cartão, dinheiro, PIX)
- **Problema que resolve:**
 - Como permitir que um objeto use diferentes algoritmos facilmente?
 - Como evitar várias condicionais (if/else) para escolher comportamentos?
 - Como trocar algoritmos em tempo de execução?

Casos de Uso para Strategy

Exemplos práticos onde o Strategy é útil:

- **Métodos de ordenação:** Bubble Sort, Quick Sort, Merge Sort
- **Métodos de pagamento:** Cartão, boleto, transferência
- **Algoritmos de compressão:** ZIP, RAR, GZ
- **Estratégias de validação:** CPF, e-mail, senha
- **Formas de navegação:** Carro, transporte público, bicicleta

Implementação do Strategy - Explicada

```
1 # Interface da estratégia - Define o contrato comum
2 from abc import ABC, abstractmethod
3
4 class OperacaoStrategy(ABC):
5     @abstractmethod
6     def executar(self, a, b):
7         # Cada estratégia concreta implementar este método
8         pass
9
10 # Estratégias concretas - Cada uma implementa o algoritmo de forma diferente
11 class SomaStrategy(OperacaoStrategy):
12     def executar(self, a, b):
13         return a + b # Implementa o para soma
14
15 class SubtracaoStrategy(OperacaoStrategy):
16     def executar(self, a, b):
17         return a - b # Implementa o para subtração
18
19 class MultiplicacaoStrategy(OperacaoStrategy):
20     def executar(self, a, b):
21         return a * b # Implementa o para multiplicação
```

Como Usar o Strategy - Passo a Passo

```
1 # Contexto - Usa a estratégia
2 class Calculadora:
3     def __init__(self, strategy=None):
4         # Inicializa com uma estratégia (opcional)
5         self.strategy = strategy
6
7     def definir_strategy(self, strategy):
8         # Permite trocar de estratégia em tempo de execução
9         self.strategy = strategy
10
11     def calcular(self, a, b):
12         # Usa a estratégia atual para realizar o cálculo
13         if self.strategy is None:
14             raise Exception("Defina uma estratégia primeiro!")
15         return self.strategy.executar(a, b)
16
17 # Uso do padrão Strategy na prática
18 # Criando as estratégias
19 soma = SomaStrategy()
20 subtracao = SubtracaoStrategy()
21 multiplicacao = MultiplicacaoStrategy()
22
23 # Criando o contexto
24 calc = Calculadora()
25
26 # Exemplo 1: Usando a estratégia de soma
27 calc.definir_strategy(soma)
28 print(f"10 + 5 = {calc.calcular(10, 5)}") # 15
29
30 # Exemplo 2: Trocando para subtração
```

Vantagens

- Troca algoritmos em tempo de execução
- Isola diferentes implementações
- Elimina condicionais complexos
- Facilita adicionar novos algoritmos
- Facilita os testes unitários

Desvantagens

- O cliente precisa conhecer as diferentes estratégias
- Aumenta o número de objetos no sistema
- Pode ser complexo demais se os algoritmos não mudam frequentemente

Perguntas para guiar a aplicação dos padrões:

- **Para Singleton:**

- Existe algo que deve ser único em todo o sistema?
- Preciso controlar o acesso a algum recurso compartilhado?

- **Para Decorator:**

- Preciso adicionar funcionalidades a objetos específicos?
- Tenho muitas combinações de características possíveis?

- **Para Strategy:**

- Existem diferentes maneiras de realizar uma mesma tarefa?
- Preciso trocar algoritmos durante a execução?

Exercício 1: Implementação de Singleton

- 1 Crie um sistema de log usando o padrão Singleton:
 - Classe: Logger (com método `log(mensagem)`)
 - A classe deve garantir que existe apenas uma instância
 - Implemente métodos para diferentes níveis: `info()`, `warning()`, `error()`
- 2 Teste o Logger em diferentes partes do código
- 3 **Demonstração:** Mostre que é a mesma instância sendo usada
- 4 **Funcionalidade adicional:** Adicione um método para salvar os logs em arquivo

Exercício 1: Esqueleto do Código

```
1 class Logger:
2     # Variavel para armazenar a instancia unica
3     _instance = None
4
5     @classmethod
6     def get_instance(cls):
7         # Seu codigo aqui: retornar a instancia existente
8         # ou criar uma nova se nao existir
9         pass
10
11     def __init__(self):
12         # Seu codigo aqui: prevenir multiplas instancias
13         self.logs = []
14
15     def log(self, message, level="INFO"):
16         # Seu codigo aqui: registrar a mensagem com timestamp
17         pass
18
19     def info(self, message):
20         # Registrar informacao
21         pass
22
23     def warning(self, message):
24         # Registrar aviso
25         pass
26
27     def error(self, message):
28         # Registrar erro
29         pass
```

-  Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
-  Freeman, E., Robson, E. (2004). Head First Design Patterns. O'Reilly Media.
-  Bloch, J. (2018). Effective Java, 3rd Edition. Addison-Wesley.
-  Refactoring.Guru - Design Patterns <https://refactoring.guru/design-patterns>

Obrigado pela atenção!

Contato: lucasbruzzzone@libertas.edu.br